Lab 3: Smaller, Faster Neural Networks with Pi

MLPs, CNNs, GNNs, sparsity, and reduced precision · 100 points

Individual assignment · Submit via Classroom50

Release and due dates: posted with the assignment

The question

How much of a trained neural network can be removed, or represented with fewer bits, before useful behavior is lost? More importantly, when do fewer parameters or operations produce an *actual* reduction in model file size or measured laptop runtime?

You will use the **Pi coding agent** to implement and execute a controlled compression study on three standard, laptop-scale reference problems:

- an **MLP** on flattened Fashion-MNIST images;
- a small **CNN** on Fashion-MNIST images; and
- a two-layer **GNN** for node classification on Cora.

Pi does the coding, testing, instrumentation, and experiment execution. You make the scientific choices, inspect the evidence, and write the final one-page implications memo. This is not a contest for the smallest model. A negative result—for example, 90% zeros with no CPU speedup—is valuable when it is measured and explained.

Learning objectives

By the end of the lab, you should be able to:

1. distinguish parameter count, nonzero count, serialized size, theoretical operations, memory use, and measured latency;
2. compare unstructured, structured, saliency-based, and semi-structured sparsity under a common protocol;
3. distinguish pruning a network's weights from changing the GNN input graph;
4. measure the accuracy–size–runtime trade-offs of reduced precision; and
5. direct a coding agent while retaining human responsibility for experimental design and scientific interpretation.

Agent/human division of labor

	Pi coding agent	You
Design	Propose implementation options and identify constraints	Freeze hypotheses, metrics, budgets, seeds, pruning levels, and stopping rules before runs
Execution	Write code/tests; download public data; train, prune, quantize, benchmark; save raw results	Review commands and diffs; approve bounded actions; independently rerun smoke tests
Interpretation	Check arithmetic and point to anomalous records	Decide what the evidence means and write the one-page memo in your own words
Provenance	Produce complete transcripts and a machine-readable run manifest	Ensure no transcript, failure, or unsuccessful arm is omitted

The one-page memo may not be drafted or rewritten by Pi or another generative model. You may ask Pi to check a number, locate the raw record supporting a claim, or identify a possible contradiction. Record that interaction in the transcript.

Common experimental contract

Use Python 3.10+ and PyTorch on CPU by default. A laptop GPU may be used only if every compared arm uses the same device; report the hardware and software versions. Set and record Python, NumPy, and PyTorch seeds. Data splits must be created once, saved, and reused by every arm.

Reference models. Use compact models that train in minutes, not state-of-the-art models. Suggested starting points are MLP $784 \to 256 \to 128 \to 10$, a two-convolution LeNet-style CNN, and a two-layer GCN on Cora. Pi may adjust these only after you record the reason. Train each dense FP32 baseline to a stable validation-selected checkpoint. As a sanity target, expect roughly 82%+ MLP accuracy and 87%+ CNN accuracy on Fashion-MNIST, and 65%+ GNN accuracy on the saved Cora split. If a baseline misses a target, diagnose it rather than silently lowering the target.

Fair comparisons. All compression arms for a model start from the same saved dense checkpoint and use the same test set. Pruning arms receive the same fine-tuning budget. Hyperparameters may differ across architectures, but not opportunistically across methods after test results are seen. Never select a model using the test set.

Metrics. For every run record: architecture, method, target and achieved sparsity or precision, seed, training/fine-tuning epochs, validation metric, test metric, total parameters, nonzero parameters, dense and compressed on-disk bytes, estimated multiply-accumulates when meaningful, peak resident memory, and latency. Measure latency after at least 20 warm-up iterations and at least 100 timed iterations; report median and interquartile range for batch size 1 and one throughput-oriented batch size. For Cora, one full-graph inference is the latency unit. Disable gradient tracking and pin the CPU thread count.

Two runtime paths. Measure (1) the ordinary dense PyTorch representation with zero-valued weights and (2) a genuinely compact or sparse exported representation when the software stack supports it. Do not infer speedup from the number of zeros. If a method has no efficient kernel on your laptop, record that as a result.

1 Part A: Three dense reference models (15 points)

Human checkpoint

Before Pi trains anything, write `predictions.md`: rank the MLP, CNN, and GNN by parameter count, dense checkpoint size, training time, and sensitivity to compression. Give a mechanism for each ranking. Freeze the data splits, architectures, maximum epochs, early-stopping rule, and three confirmatory seeds.

Delegate to Pi

Ask Pi to create a single command-line experiment package with model/dataset flags, deterministic splits, unit tests, checkpointing, CSV/JSONL logging, and a benchmark command. Require one tiny overfit test per architecture, a parameter-count test, and a test proving that evaluation never updates weights.

Required evidence

Three validated dense checkpoints; learning curves; baseline metrics and latency table; split files; tests and test output; exact replay commands. Pi's transcript must show your initial prompt, its edits, commands, failures, and corrections.

2 Part B: Four forms of sparsity (40 points)

Sparsity applies to **learned network weights or groups**, not to deleting Cora edges. Implement all four methods below for all three architectures where the method is well-defined.

S1. Global unstructured magnitude pruning. Across eligible weight tensors, remove the weights with smallest $|w|$. Biases and normalization parameters are excluded.

S2. Structured group pruning. Remove complete MLP hidden units, CNN output channels, or GNN hidden features. Rank groups by the L_1 norm of their weights and physically rebuild a smaller dense model so that parameter and operation reductions are real.

S3. First-order saliency pruning. Using a fixed calibration subset, rank eligible weights or groups by a first-order criterion such as $|w\,\partial L/\partial w|$. State whether the implementation is unstructured or group-structured and keep that choice fixed.

S4. N:M semi-structured pruning. Within each consecutive block of M eligible weights, retain the N largest magnitudes. Use 2:4 plus at least one less aggressive setting, or document why the local backend cannot represent one setting.

For S1–S4, test target sparsities of 25%, 50%, 75%, and 90% when mathematically possible. Use one frozen pilot seed for the complete $3 \times 4 \times 4$ matrix. Then choose, by a validation-only rule written in advance, one useful operating point per architecture/method and repeat that point with all three confirmatory seeds. Each pruned model receives the same fixed fine-tuning budget. Save measurements both immediately after pruning and after fine-tuning.

Human checkpoint

Before each method runs, predict: (i) which architecture will lose the most accuracy at 75% sparsity, (ii) how much fine-tuning will recover, and (iii) whether parameter reduction will yield file-size and runtime reduction. Specify the validation-only rule for selecting confirmatory points. Do not change it after seeing test results.

Delegate to Pi

Ask Pi to implement masks and physical model surgery separately, verify achieved sparsity tensor by tensor, preserve a reversible mapping to the dense checkpoint, and emit one row per run. Require tests that pruned entries remain zero during fine-tuning, structured shapes are consistent, and reloaded checkpoints reproduce logits and metrics.

Required evidence

A complete pilot matrix (48 compressed configurations plus the three dense baselines), pre/post-fine-tuning records, three-seed confirmation records, accuracy-versus-sparsity and latency-versus-accuracy plots, artifact sizes, and per-layer sparsity tables. Every missing or failed run remains in the manifest with a classified reason.

3 Part C: Reduced precision (25 points)

Starting from each dense baseline, compare FP32 with the formats below. Use post-training conversion first; optional quantization-aware training may be added as a labeled extension.

1. FP16 and/or BF16, choosing only formats supported by the laptop backend and reporting any automatic upcasting;
2. dynamic INT8 quantization where supported;
3. calibrated static INT8 quantization where supported; and
4. a simulated weight-only INT4 condition: pack or accurately account for 4-bit weights, dequantize for computation if no native kernel exists, and label runtime as *simulated/non-native*.

Use one fixed calibration subset selected without test labels. Record the calibration size and selection rule. Report accuracy, actual serialized bytes, latency, memory, and the fraction of operations that truly ran at reduced precision. A renamed dtype or theoretically smaller tensor is not evidence of a faster model.

After the standalone precision study, select **one sparse operating point per architecture** using validation data and combine it with the best supported INT8 method. Compare dense FP32, sparse FP32, dense INT8, and sparse+INT8. This small factorial comparison tests whether the benefits compose.

Human checkpoint

Predict separately which format will minimize file size and which will minimize latency. Explain why those need not be the same. Freeze the calibration rule and the sparse operating points before test evaluation.

Delegate to Pi

Ask Pi to inspect backend support, implement the conversions, validate reload equivalence within an explicit tolerance, and benchmark native execution. Require the results schema to distinguish stored precision, compute precision, and accumulator precision.

Required evidence

Precision-support report; calibration record; all standalone precision results; the 12-condition sparse/precision factorial comparison; size and latency plots; reload/tolerance tests; classified unsupported cases.

4 Part D: Human implications memo (20 points)

Write a **single page**, maximum 650 words, with one compact figure or table permitted. Use your own prose and address all of the following:

1. Which compression method produced the best trade-off for each architecture, and what evidence supports that claim?
2. When did fewer parameters/nonzeros fail to make inference faster, and why?
3. How did architecture change sensitivity to sparsity and reduced precision?
4. Did sparsity and INT8 benefits compose, interfere, or depend on backend support?
5. What would you recommend for deploying each model on a laptop, and what important uncertainty remains?

Every quantitative claim must point to a run ID in parentheses. End with a three-sentence reflection: one decision Pi made well, one decision you overruled or corrected, and one conclusion that required human judgment rather than additional code.

Submission package

Submit `report.pdf` separately and `lab3_CNetID.zip` with:

Path	Required contents
`README.md`	Name/CNetID, hardware/software, total compute time, setup, and one replay command per part
`memo.pdf`	The one-page, human-authored implications memo
`predictions.md`	Dated predictions and frozen design decisions written before corresponding runs
`code/`, `tests/`	Pi-produced implementation and tests
`configs/`, `splits/`	Every configuration and the exact saved train/validation/test splits
`results/runs.jsonl`	One immutable record per attempted run, including failures and unique run IDs
`checkpoints/`	Dense baselines and selected compressed exports; omit redundant large intermediates
`figures/`	Figures used in the memo plus the required trade-off plots
`transcripts/`	Complete, unedited Pi transcript(s), including prompts, tool calls, errors, and corrections
`AGENT_DISCLOSURE.md`	What Pi implemented, what you specified, and all other assistance used
`MANIFEST.sha256`	Hashes of submitted files

Rubric summary

Dense references: reproducibility, tests, fair baselines	15
Four sparsity methods: correct implementation and complete controlled matrix	20
Sparsity evidence: real size/runtime measurements, confirmations, failure accounting	20
Reduced precision: correct support/calibration study and sparse+INT8 comparison	25
One-page human memo: evidence, mechanisms, judgment, clarity	15
Provenance: predictions, transcripts, disclosure, replay instructions, manifest	5
Total	100

Scoring principles. Evidence beats agent narrative. Unsupported backends and negative speedups can receive full credit when tested honestly. Fabricated runs, altered test results, omitted failures, test-set model selection, or agent-written memo prose invalidate the affected credit. The goal is a defensible experiment, not a predetermined result.

Pre-submission checks

- ☐ All three dense baselines pass tests and use frozen splits.
- ☐ The 48-arm sparsity pilot matrix is complete or every missing arm has a recorded failure reason.
- ☐ Confirmatory operating points use three seeds and were selected using validation data only.

- □ Latency reports warm-up, repetition count, thread count, batch/unit, median, and IQR.
- □ Stored precision, compute precision, and accumulator precision are not conflated.
- □ The one-page memo is at most 650 words and was written by the student.
- □ Every memo number names a run ID; the archive includes raw records and full Pi transcripts.